



CERNE

GB AGRITECH · UX & DESIGN SYSTEM

Processo de UX Code-First

Design System, Tokens e Desenvolvimento Assistido por IA

O novo processo de UX da GB Agritech, fundamentado no projeto CERNE.

Autoria	Arthur Meireles — UX, GB Agritech
Base técnica	Projeto CERNE (React + TypeScript + shadcn/ui + tokens)
Versão	1.0 — Junho/2026
Classificação	Documento técnico interno — Confidencial

Em uma frase

Adotar o código como fonte única da verdade do design system — tokens versionados, componentes distribuídos por registry privado e padrões de composição legíveis por IA — transformando o handoff tradicional em um processo de UX ágil onde o próprio sistema alimenta a geração de código assistida por IA. Esta abordagem não é uma aposta: é o padrão consolidado da indústria em 2025–2026.

SEÇÃO 1

Sumário Executivo

A GB Agritech está migrando um produto existente para uma nova interface (projeto CERNE) e, no mesmo movimento, redefinindo como UX e Engenharia colaboram. O processo tradicional — desenhar no Figma e repassar para os desenvolvedores implementarem — foi invertido: o código é a fonte da verdade, e o Figma deixa de ser entregável obrigatório.

Este documento consolida o desenho do novo processo e o fundamenta na pesquisa do estado atual da indústria. A conclusão central é que a arquitetura proposta está alinhada ao padrão consolidado de 2025–2026, e que seu maior valor não é organizacional, mas estratégico: um design system code-first bem governado é hoje a pré-condição para que a IA gere código confiável em escala.

Os quatro pilares do processo

- Tokens como constituição — valores de design (cor, espaço, tipografia) vivem em código, versionados, no padrão W3C Design Tokens. Fonte única para todos os produtos da GB.
- Registry privado de componentes — primitivas curadas distribuídas via registry shadcn privado; os times consomem por linha de comando e mantêm o código no próprio repositório.
- Governança federada — UX é a fonte central dos tokens; desenvolvedores criam componentes livremente, e os melhores são promovidos ao núcleo via engenharia reversa e aprovação de UX.
- Contexto para IA — as leis e receitas de composição são empacotadas como contexto legível por agentes (Claude), permitindo construir telas novas com consistência mesmo sem Figma.

SEÇÃO 2

Contexto e Problema

Situação atual

Hoje o fluxo de UX termina no código (com alta fidelidade e governança via as Leis do projeto CERNE) e só então é convertido para o Figma — um passo manual, custoso e que quebra os vínculos com o design system. O resultado é um Figma que funciona como espelho desatualizado de algo que já existe melhor no código, gerando a percepção (correta) de baixo valor.

O desafio da migração

- Durante a migração, a tela antiga do produto existente é a especificação funcional — há referência de conteúdo e comportamento, ainda que não exista Figma.
- Após a migração, os desenvolvedores criarão telas inteiramente novas, sem mockup nem referência visual. É nesse momento que a consistência precisa ser garantida por outro mecanismo.

O risco central a evitar

No instante em que se regenera em vez de distribuir, bifurca-se a fonte da verdade. O valor de um design system é “uma definição, muitos consumidores”. Sem um mecanismo de re-centralização e verificação, o resultado não é um design system, mas N sistemas parecidos que divergem a cada sprint. Toda a arquitetura abaixo existe para preservar essa fonte única.

Como o stack dos desenvolvedores é o mesmo (React + shadcn/ui reescrito com os tokens da GB), a estratégia correta é distribuir e governar, não redesenhar — e usar a IA como aceleradora de composição sobre uma base canônica.

SEÇÃO 3

Estado da Indústria (2025–2026)

A pesquisa em fontes do setor confirma que as três grandes escolhas do processo — tokens code-first, registry privado e governança via lint — são hoje o mainstream, e duas delas ganharam suporte de primeira classe há poucos meses.

3.1 Code-first venceu — e o W3C selou o padrão

A pergunta “Figma ou código?” está resolvida na prática: tokens vivem em JSON versionado no repositório, e o Figma consome de lá. Em outubro de 2025 o W3C publicou a primeira especificação estável de Design Tokens (DTCG, v2025.10), já implementada por mais de 10 ferramentas (Figma, Tokens Studio, Style Dictionary). A busca por “design tokens generator” cresceu mais de 900% em dois anos.

3.2 Registry privado é feature de primeira classe

O shadcn CLI 3.0 (agosto de 2025) introduziu registries nomeados e privados: empresas servem seus componentes proprietários via `@gb/componente`, com autenticação (bearer token, API key) e resolução automática de dependências — descrito como “perfect for enterprise teams with proprietary UI libraries”.

3.3 Governança é o problema difícil, não a ferramenta

O modo de falha universal: alguém altera um token diretamente e, em um dia, código e design divergem — “não por descuido, mas porque não há regra de onde a mudança deve originar”. A resposta da indústria é propriedade explícita somada a automação (lint/CI) até o ponto em que apenas a revisão humana é necessária.

SEÇÃO 4

O Encaixe com Inteligência Artificial

A tese do processo — “o design system alimenta a IA” — é literalmente o consenso de 2026, e a própria Figma o admite.

“AI strengthens your design system, which powers better AI code generation.”

— Figma, Design Systems and AI: Why MCP Servers Are the Unlock

O efeito flywheel

Um design system robusto gera melhor código com IA, que por sua vez reforça o sistema. O dado que justifica o investimento: apenas 32% dos profissionais confiam no output de IA sem contexto de design. Sem tokens, código gerado por IA é inconsistente por padrão.

Três fatos que potencializam o processo da GB

- O registry shadcn é nativamente consumível por agentes — o CLI 3.0 trouxe um servidor MCP com `search / view / list / add`. O Claude dos desenvolvedores pode descobrir e instalar componentes do registry GB de forma autônoma.
- Camada de contexto para agentes — o padrão emergente é fornecer contexto file-based, legível e versionado junto ao código (estilo AGENTS.md). O CLAUDE.md do CERNE, com as Leis e as Regras de composição, já é exatamente isso.
- Cautela sobre MCP — há benchmarks mostrando custo de contexto elevado em operações MCP (~32k–82k tokens) frente a CLIs leves (~200 tokens). Recomenda-se começar com CLI + skill e adotar MCP apenas se a fricção justificar.

SEÇÃO 5

Arquitetura-Alvo

A divisão entre pacote (npm) e registry encaixa diretamente na decisão de “compartilhar mais tokens do que componentes”: os tokens são uma dependência travada e central; os componentes são um commons curado que os times possuem e estendem.

```
@gb/tokens (npm | semver | TRAVADO)          <- a constituicao: 1 definicao p/ toda a GB
| Tailwind preset + CSS vars + tokens.json (DTCG)
v
Registry shadcn privado GB (pull & own)      <- primitivas curadas (nucleo)
|
v
Projetos GB (migracao + futuros)             <- devs criam componentes livremente
| ^ enforcement: @gb/lint (proibe hardcode) no CI <- o controle da federacao
| ^ Claude: skill gb-design-system (receitas de composicao) <- substitui o Figma
v
Pipeline de promocao (engenharia reversa)
+- dev cria -> UX aprova -> tokeniza/normaliza -> publica no registry
```

As camadas

Camada	Formato	Modelo	Quem controla
Tokens	@gb/tokens (npm, semver)	Dependência travada; atualização central propaga	UX — estrito
Primitivas núcleo	Registry shadcn privado	Pull & own (filosofia shadcn)	UX cura; devs consomem
Componentes de projeto	Código no repo do dev	Federado, livre	Devs criam; UX promove
Enforcement	@gb/lint (npm) no CI	Obrigatório em todo repo GB	UX define a regra
Receitas de tela	Skill gb-design-system	Claude carrega sob demanda	UX mantém

SEÇÃO 6

Plano de Implementação

Seis frentes, da pedra angular às contínuas. A sequência recomendada é 1 → 2 → 3+4 em paralelo → 5 e 6 contínuas. A frente 1 destrava todas as outras.

Frente 1 — @gb/tokens — a pedra angular

tokens.ts → tokens.json (DTCG) → Style Dictionary gera o Tailwind preset e as CSS vars. Todo projeto da GB faz extend do mesmo preset — mesma cor, espaço, raio e tipografia, por construção. Publicado como pacote npm versionado.

Frente 2 — @gb/lint — o controle da federação

A Lei de tokenização (nada de hex/px fora dos tokens) vira um ESLint/Stylelint config obrigatório no CI de todo repositório GB. É o que permite criar componentes à vontade sem o sistema desandar: o que se cria é forçado a falar a língua dos tokens.

Frente 3 — Registry shadcn privado — núcleo curado

Registry custom (registry.json + JSON por componente) servido em uma URL autenticada. Os devs configuram o components.json apontando para o registry GB e instalam com `npx shadcn add @gb/button`. Semear com os mais reusados (Button, FormField, FormSelect, Card, Badge, DataTable, Modal, PageCard).

Frente 4 — Skill gb-design-system — o que substitui o Figma

Empacotar as Leis, as Regras de composição (listagem e CRUD), os contratos de componente e a referência de tokens. O Claude de qualquer dev carrega a skill e monta telas novas sem Figma, compondo primitivas do registry e respeitando os tokens. Versionada junto com os tokens.

Frente 5 — Pipeline de promoção — a engenharia reversa formalizada

- Dev cria o componente no projeto.
- UX revisa contra critérios fixos: tokenizado, dois temas, story, acessibilidade, todos os estados.
- Normalização (o Claude pode auditar e propor a versão tokenizada — automatizável).
- Publicação no registry → vira commons compartilhado.

Frente 6 — A migração

A tela antiga é a especificação (conteúdo e comportamento). Dev + Claude + skill + registry reconstroem cada tela. Telas pós-migração apoiam-se 100% nas receitas. A migração alimenta o registry: componentes novos que surgem viram candidatos a núcleo.

SEÇÃO 7

Governança e Modelo de Contribuição

Como o CERNE se torna o padrão de toda a GB Agritech, a governança deixa de ser documento e vira produto versionado. O modelo é federado: tokens centralizados e estritos; componentes flexíveis na borda, com um caminho de promoção ao núcleo.

Controles obrigatórios

Controle	O que faz	Modo de falha que previne
Token-lint no CI	Bloqueia valores hardcoded fora dos tokens	Drift visual entre produtos
Propriedade explícita	UX é dono dos tokens; devs donos dos componentes de projeto	“Quem percebeu” conserta
Versionamento (semver)	Mudança de token = bump = changelog = propaga	Atualizações silenciosas e quebras
Pipeline de promoção	Critério fixo para um componente virar núcleo	Sprawl de componentes redundantes
AI-linting (evoluir p/)	Agente audita PRs contra as Leis antes da revisão humana	Revisão manual cara e tardia

Reposicionamento de discurso

Internamente na GB, vender isto não como “tooling de UX”, mas como a pré-condição para que a IA gere código confiável em escala — que é, segundo a própria Figma e o consenso de 2026, o real destravamento de produtividade.

SEÇÃO 8

Ajustes Recomendados pela Pesquisa

A pesquisa valida a arquitetura; os pontos abaixo são refinamentos, não correções.

❶ — Adotar o formato W3C DTCG explicitamente

Gerar o `tokens.json` no padrão DTCG (`$value / $type`) garante portabilidade entre mais de 10 ferramentas e mantém a porta do Figma aberta como consumidor downstream (via Tokens Studio), sem custo extra. O mesmo arquivo alimenta Style Dictionary e Tailwind preset e, se desejado, o Figma.

❷ — Aproveitar o shadcn MCP, mas começar leve

O registry já é AI-discoverable, o que potencializa a federação. Porém, dado o custo de contexto do MCP, recomenda-se começar com CLI + skill `gb-design-system` (leve e suficiente) e adicionar o servidor MCP apenas se a fricção de descoberta justificar. Não construir MCP por construir.

❸ — Tratar governança como produto versionado

O token-lint no CI deixa de ser “nice to have” e vira o controle central. Vale um passo além: AI-linting — um agente que audita PRs contra as Leis (hardcode, fonte errada, primitiva reinventada) e sinaliza antes da revisão de UX. A indústria caminha para bots verificando compliance em tempo real.

SEÇÃO 9

Riscos e Controles

Risco	Impacto	Controle
Bifurcação da fonte da verdade	N design systems divergentes	Tokens centrais + versionamento + lint
Sprawl de componentes	Redundância e inconsistência	Pipeline de promoção + “consultar registry antes de criar”
Drift de tokens	Inconsistência visual entre produtos	@gb/lint obrigatório no CI
Dependência do operador (bus-factor)	Conhecimento na cabeça do UX	Leis e processo documentados + Storybook publicado
Over-engineering de IA/MCP	Custo de contexto e manutenção	Começar com CLI + skill; MCP sob demanda

Métricas de saúde sugeridas

- Cobertura de tokens (% de valores de design vindos de t . *) por repositório.
- Tempo médio de promoção de um componente (criação → publicação no registry).
- N° de violações de token-lint barradas no CI por sprint (tendência decrescente).
- Adoção do registry (n° de produtos GB consumindo @gb/tokens e o registry).

SEÇÃO 10

Glossário

Termo	Definição
Design token	Valor nomeado de design (cor, espaço, raio, tipografia) que substitui literais hardcoded.
W3C DTCG	Design Tokens Community Group; formato-padrão (JSON com \$value/\$type) estável desde out/2025.
SSOT	Single Source of Truth — a fonte única e autoritativa de uma definição.
shadcn registry	Mecanismo do shadcn para distribuir componentes como JSON via CLI; suporta registries privados.
Style Dictionary	Ferramenta que transforma um arquivo de tokens em múltiplos formatos (CSS, Tailwind, etc.).
Tokens Studio	Plugin Figma que sincroniza variáveis com um arquivo de tokens em Git.
MCP	Model Context Protocol — protocolo que dá a agentes de IA contexto e ferramentas estruturadas.
Token-lint	Regra de lint que falha o build quando há valores de design hardcoded fora dos tokens.
Federação	Modelo onde times criam componentes localmente, com promoção curada ao núcleo compartilhado.
Flywheel	Ciclo virtuoso: o design system melhora a IA, que reforça o design system.

SEÇÃO 11

Referências

1. Figma — Design Systems and AI: Why MCP Servers Are the Unlock
<https://www.figma.com/blog/design-systems-ai-mcp/>
2. shadcn/ui — CLI 3.0 and MCP Server (changelog, ago/2025)
<https://ui.shadcn.com/docs/changelog/2025-08-cli-3-mcp>
3. shadcn/ui — Registry (documentação)
<https://ui.shadcn.com/docs/registry>
4. Design Token SSOT: Figma or Code? The Governance Question (and AI's Role) — Design Systems Collective
<https://www.designsystemscollective.com/design-token-ssot-figma-or-code-the-governance-question-and-ais-role-6dc6dfda533c>
5. Figma is not the source of truth for design tokens — Artur Sopelnik
<https://artursopelnik.de/posts/figma-is-not-the-source-of-truth/>
6. Design Tokens in 2026: Auto-Generate Them in Seconds — OneMinuteBranding
<https://www.oneminutebranding.com/blog/design-tokens-2026>
7. Figma is no longer the source of truth — JumpCloud Design
<https://medium.com/@jc-design/figma-is-no-longer-the-source-of-truth-adb89feabafb>
8. My Predictions for MCP and AI-Assisted Coding in 2026 — DEV Community
<https://dev.to/blackgirlbytes/my-predictions-for-mcp-and-ai-assisted-coding-in-2026-16bm>
9. AI Engineering Trends in 2025: Agents, MCP and Vibe Coding — The New Stack
<https://thenewstack.io/ai-engineering-trends-in-2025-agents-mcp-and-vibe-coding/>

Documento gerado a partir de pesquisa de fontes do setor (2025–2026) e do desenho de processo validado com a equipe de UX da GB Agritech. As datas de acesso às fontes correspondem a junho/2026.